

BACKEND ENGINEERING · LILISHOP

The Result Pattern

A Practical Guide

A clear, beginner-friendly guide to the Result Pattern: what it is, why it exists, how it is built, and how to correctly handle the confusing cases — null, 0, and empty collections.

PREPARED BY

Said Roohullah Allem<https://github.com/jahanalem>

Contents

1. What Is the Result Pattern?	4
2. Why Do We Use It?	4
Problem 1 — Exceptions are the wrong tool for *expected* situations	4
Problem 2 — "Not found" should not always be an error	5
Problem 3 — Consistency between services and the API	5
3. The Mental Model: Three Outcomes	5
4. The Building Blocks	6
5. The Full Implementation	7
5.1 OperationResultStatus	7
5.2 ErrorCode	7
5.3 OperationResult and OperationResult<T>	8
5.4 DefaultMessages	13
5.5 ErrorCodeToStatusMapper	14
5.6 OperationResultHandler	15
5.7 BaseApiController	17
6. Responsibilities at a Glance	17
7. How to Use It in Practice	18
7.1 Creating results in a service	18
7.2 Consuming another service's result (service-to-service)	19
7.3 Using results in a controller	20
8. The Tricky Part: null, 0, and Empty Collections	21
8.1 An empty collection → Success(emptyList) → 200	21
8.2 The number 0 → Success(0) → 200	22
8.3 The boolean false → Success(false) → 200	22
8.4 A missing single resource → NoData<T>() → 204	22
8.5 null data → not allowed (throws)	23
8.6 The whole decision in one table	24
9. How Results Become HTTP Responses	24
10. Quick Reference Cheat Sheet	25
11. Common Mistakes (FAQ)	26
12. Summary	26

1. What Is the Result Pattern?

When a method does some work, it can end in different ways:

- It **worked** and produced a value.
- It **worked** but there was nothing to return.
- It **failed** for a known reason (not found, invalid input, a database error...).

The **Result Pattern** is a simple idea: instead of **only** returning the value (or throwing an exception for every problem), a method returns a small **object that describes the outcome**.

That object answers three questions:

1. Did it succeed or fail?
2. If it succeeded, what is the data?
3. If it failed, why? (a machine-readable error code and a human-readable message)

In this codebase that object is called `OperationResult` (for operations with no data) and `OperationResult<T>` (for operations that return data of type `T`).

```
// Instead of this (throws on every problem):
ProductBrand brand = await GetBrand(id); // throws if not found? returns null? who knows.

// We do this (the outcome is explicit):
OperationResult<ProductBrand> result = await GetBrandByIdAsync(id);
if (result.IsSuccess) { /* use result.Data */ }
else { /* handle result.ErrorCode / result.Message */ }
```

The same result object is used **everywhere** — both when one service calls another service, and when a controller turns the outcome into an HTTP response. One consistent shape, end to end.

2. Why Do We Use It?

Problem 1 — Exceptions are the wrong tool for **expected** situations

Exceptions are great for **unexpected** problems (the database is down, a bug, a null reference). But "the user asked for a brand that does not exist" is **not unexpected** — it happens all the time. Using exceptions for normal, expected outcomes is slow, noisy, and easy to forget to catch.

A result object makes the expected outcomes **part of the method's return value**, so the caller **cannot ignore them by accident**.

Problem 2 — "Not found" should not always be an error

Imagine a page that asks: *"Is there a discount for this product?"*

If the answer is "no discount," that is **not an error**. If the API returned `404 Not Found`, the frontend might pop up an error dialog — which is wrong, because nothing went wrong.

The Result Pattern lets us say: *"The operation succeeded; there is simply nothing to show."* The API then returns `204 No Content` instead of `404`, and the frontend treats it as an empty, normal outcome.

Problem 3 — Consistency between services and the API

Because every service returns the same kind of object, we get:

- **Loose, predictable communication** between services (no guessing what `null` means).
- **One place** (the API handler) that decides how an outcome becomes an HTTP status code.
- **Easy testing**: a fake service just returns `OperationResult.Success(...)` or

`OperationResult.Failure(...)` — no special setup needed.

3. The Mental Model: Three Outcomes

Everything in the pattern comes down to **three** possible outcomes. If you remember only this table, you understand 90% of the pattern:

Outcome	Meaning	Typical HTTP result
Success	The operation worked and (optionally) produced data .	<code>200 OK</code> / <code>204 No Content</code>
NoData	The operation worked, but the requested resource does not exist .	<code>204 No Content</code>
Failure	The operation failed for a known reason.	<code>4xx</code> / <code>5xx</code> (from the error code)

These three are represented by the `OperationResultStatus` enum.

The single most important rule: NoData means *"a specific resource is missing"* — like a lookup by id that found nothing. It does **not** mean "an empty list," "the number zero," or "false." Those are **valid data** and use **Success**. (Section 8 explains this in detail.)

4. The Building Blocks

The pattern is made of a few small, focused pieces:

File / Type	Layer	Responsibility
<code>OperationResultStatus</code>	Application	The enum of the three outcomes (Success / NoData / Failure).
<code>ErrorCode</code>	Application	A list of known failure reasons (machine-readable).
<code>OperationResult</code>	Application	The result object for void operations + the factory methods .
<code>OperationResult<T></code>	Application	The result object that also carries data of type T.
<code>DefaultMessages</code>	Application	A safe default text message for each <code>ErrorCode</code> .
<code>ErrorCodeToStatusMapper</code>	Infrastructure	Translates an <code>ErrorCode</code> into an HTTP status code.
<code>OperationResultHandler</code>	Infrastructure	Turns an <code>OperationResult</code> into an <code>ActionResult</code> (the HTTP response).

File / Type	Layer	Responsibility
BaseApiController	API	A tiny helper so controllers can call the handler easily.

The first five live in the **Application** layer (the core, framework-independent part). The last three live closer to the web (they know about HTTP). This keeps your business logic free of HTTP concerns.

5. The Full Implementation

This is the complete, real implementation used in the project.

5.1 OperationResultStatus

```
namespace LiliShop.Application.Common.Results
{
    /// <summary>
    /// The outcome category of an <see cref="OperationResult"/>.
    /// </summary>
    public enum OperationResultStatus
    {
        /// <summary>The operation succeeded and produced a result.</summary>
        Success,

        /// <summary>
        /// The operation succeeded but the requested resource does not exist (e.g. a
        lookup that matched
        /// nothing). The API layer maps this to 204 No Content so "not found" does not
        surface as a 404 error.
        /// This is distinct from a <see cref="Success"/> carrying an empty collection,
        <c>0</c> or <c>>false</c>,
        /// which are valid data returned as 200 OK.
        /// </summary>
        NoData,

        /// <summary>The operation failed; see <see cref="OperationResult.ErrorCode"/> for
        the reason.</summary>
        Failure
    }
}
```

5.2 ErrorCode

A simple enum of **known** failure reasons. Services return one of these; the API maps it to an HTTP status. (The real file documents each value with XML comments — values are shown grouped here for brevity.)

```
namespace LiliShop.Application.Common.Results
{
    public enum ErrorCode
    {
        // General
        GeneralException,
        InvalidArgument,
        InvalidData,
        SettingError,

        // Not found
        ResourceNotFound,
        FileNotFound,
        UserNotFound,

        // Conflict
        ResourceAlreadyExists,

        // Operation failures
        AddOrUpdateFailed,
        DeletionFailed,
        DataFetchFailed,
        SaveOperationFailed,
        EmailSendFailed,
        NotificationSendFailed,
        UpdateOperationFailed,
        PhotoUploadFailed,
        MappingFailed,
        CreationFailed,
        UpdateFailed,

        // External services
        StripeError,

        // Auth
        AuthorizationRequired,
        TokenExpired,
        InvalidRefreshToken,
        UserNotFoundForRefreshToken,
        InvalidPassword
    }
}
```

5.3 OperationResult and OperationResult<T>

This is the heart of the pattern. Notice there are **no public constructors** — you create results only through the **static factory methods** ([Success](#), [NoData](#), [Failure](#)). This keeps creation consistent and lets us enforce rules (like "Success can never carry `null`").

```
namespace LiliShop.Application.Common.Results
{
    /// <summary>
    /// Represents the outcome of an operation. The same type is used for both service-to-
    service
    /// calls and the API layer, so a result can be consumed directly (via <see
    cref="IsSuccess"/>,
    /// <see cref="IsFailure"/> and <see cref="ErrorCode"/>) or translated into an HTTP
    response by the
    /// API layer.
    /// <para>
    /// Create instances through the static factory methods (<see cref="Success()"/>,
    /// <see cref="NoData()"/>, <see cref="Failure(Results.ErrorCode, string?)" /> and
    their generic
    /// overloads) rather than constructors.
    /// </para>
    /// </summary>
    public class OperationResult
    {
        /// <summary>The status of the operation: success, success-with-no-data, or
        failure.</summary>
        public OperationResultStatus Status { get; }

        /// <summary>A human-readable message describing the outcome.</summary>
        public string Message { get; }

        /// <summary>The error code when the operation failed; <c>null</c>
        otherwise.</summary>
        public ErrorCode? ErrorCode { get; }

        /// <summary>
        /// <c>true</c> when the operation did not fail — i.e. <see
        cref="OperationResultStatus.Success"/>
        /// or <see cref="OperationResultStatus.NoData"/>.
        /// </summary>
        public bool IsSuccess => Status != OperationResultStatus.Failure;

        /// <summary><c>true</c> when the operation failed.</summary>
        public bool IsFailure => Status == OperationResultStatus.Failure;

        private protected OperationResult(OperationResultStatus status, string message,
        ErrorCode? errorCode)
        {
            Status = status;
            Message = message;
            ErrorCode = errorCode;
        }
    }
}
```

```

// -----
// Void operations (no payload)
// -----

/// <summary>A successful operation that returns no payload (maps to 204 No
Content).</summary>
public static OperationResult Success(string message = "Operation completed
successfully.")
    => new(OperationResultStatus.Success, message, null);

/// <summary>
/// A successful operation with nothing to return – the requested resource does
not exist, or there
/// was nothing to act on. Maps to 204 No Content, keeping "not found" off the 404
path.
/// </summary>
public static OperationResult NoData(string message = "No data was found.")
    => new(OperationResultStatus.NoData, message, null);

/// <summary>A failed operation. The HTTP status is derived from <paramref
name="errorCode"/>.</summary>
public static OperationResult Failure(ErrorCode errorCode, string? message = null)
    => new(OperationResultStatus.Failure, message ??
DefaultMessages.For(errorCode), errorCode);

// -----
// Data operations (payload of type T: object, list, bool, number, ...)
// -----

/// <summary>
/// A successful operation carrying non-null <paramref name="data"/> (maps to 200
OK).
/// An empty collection, <c>0</c> and <c>>false</c> are all valid data and are
returned as-is
/// (e.g. an empty list becomes <c>200 []</c>).
/// </summary>
/// <exception cref="ArgumentNullException">
/// Thrown when <paramref name="data"/> is <c>>null</c>. A null payload almost
always signals a bug
/// (e.g. a failed mapping); to represent a legitimately absent resource use
/// <see cref="NoData{T}(string)"/> instead, which maps to 204 No Content.
/// </exception>
public static OperationResult<T> Success<T>(T data, string message = "Operation
completed successfully.")
{
    if (data is null)
    {
        throw new ArgumentNullException(nameof(data),
            "Success requires a non-null payload. Use OperationResult.NoData<T>()
to represent an absent resource.");
    }
}

```

```

        return new OperationResult<T>(OperationResultStatus.Success, data, message,
null);
    }

    /// <summary>
    /// A successful operation whose requested resource does not exist (e.g. a lookup
by id/slug that
    /// matches nothing). Maps to 204 No Content, keeping "not found" off the 404
path.
    /// <para>
    /// This is NOT for empty collections, <c>0</c> or <c>>false</c> – those are valid
data and must be
    /// returned with <see cref="Success{T}(T, string)"/>.
    /// </para>
    /// </summary>
    public static OperationResult<T> NoData<T>(string message = "The requested
resource does not exist.")
        => new(OperationResultStatus.NoData, default!, message, null);

    /// <summary>A failed operation of type <typeparamref name="T"/>.</summary>
    public static OperationResult<T> Failure<T>(ErrorCode errorCode, string? message =
null)
        => new(OperationResultStatus.Failure, default!, message ??
DefaultMessages.For(errorCode), errorCode);

    /// <summary>
    /// Propagates an existing failure as a typed result, preserving its error code
and message.
    /// Handy when a service forwards another service's failure: <c>return
OperationResult.Failure<T>(other);</c>
    /// </summary>
    public static OperationResult<T> Failure<T>(OperationResult failed)
        => new(OperationResultStatus.Failure, default!, failed.Message,
failed.ErrorCode ?? Results.ErrorCode.GeneralException);
    }

    /// <summary>
    /// Represents the outcome of an operation that returns data of type <typeparamref
name="T"/>.
    /// </summary>
    /// <typeparam name="T">The type of the returned data.</typeparam>
    public sealed class OperationResult<T> : OperationResult
    {
        /// <summary>
        /// The data produced by the operation. Guaranteed non-null when <see
cref="HasData"/> is <c>>true</c>;
        /// the default of <typeparamref name="T"/> on <see
cref="OperationResultStatus.NoData"/> or
        /// <see cref="OperationResultStatus.Failure"/>.
        /// </summary>
        public T Data { get; }
    }

```

```
    /// <summary>
    /// <c>true</c> only when the operation succeeded with a payload
    /// (<see cref="OperationResultStatus.Success"/>), in which case <see
    cref="Data"/> is non-null.
    /// <c>false</c> for <see cref="OperationResultStatus.NoData"/> and
    /// <see cref="OperationResultStatus.Failure"/>. Prefer this over null-checking
    <see cref="Data"/>
    /// in service-to-service code.
    /// </summary>
    public bool HasData => Status == OperationResultStatus.Success;

    internal OperationResult(OperationResultStatus status, T data, string message,
        ErrorCode? errorCode)
        : base(status, message, errorCode)
    {
        Data = data;
    }
}
```

Key properties you will use every day:

Member	On which type	What it tells you
Status	both	The raw outcome (Success / NoData / Failure).
Message	both	A human-readable description.
ErrorCode	both	The failure reason (null when not a failure).
IsSuccess	both	true for Success or NoData (i.e. "did not fail").
IsFailure	both	true only for Failure.
Data	OperationResult<T>	The payload (only meaningful on Success).

Member	On which type	What it tells you
HasData	OperationResult<T>	true only on Success — and then Data is guaranteed non-null.

5.4 DefaultMessages

If you create a failure without a custom message, this provides a safe, user-friendly default text.

```
namespace LiliShop.Application.Common.Results
{
    /// <summary>
    /// Provides the default, user-safe message for each <see cref="ErrorCode"/>.
    /// Used when a failure result is created without an explicit message.
    /// </summary>
    public static class DefaultMessages
    {
        public static string For(ErrorCode errorCode)
        {
            return errorCode switch
            {
                ErrorCode.GeneralException => "An unexpected error occurred, please try
again.",
                ErrorCode.InvalidArgument => "The provided argument is invalid. Please
check your input and try again.",
                ErrorCode.InvalidData => "The data provided is invalid or
corrupted.",
                ErrorCode.ResourceNotFound => "The requested resource could not be
found.",
                ErrorCode.UserNotFound => "The specified user does not exist or could
not be found.",
                ErrorCode.ResourceAlreadyExists => "The resource already exists and cannot
be created again.",
                ErrorCode.DeletionFailed => "Failed to delete the resource. It may be in
use or protected.",
                ErrorCode.UpdateFailed => "Failed to update the resource, please try
again.",
                // ... one arm per ErrorCode ...
                _ => "An unknown error has occurred. Please contact support if the problem
persists."
            };
        }
    }
}
```

5.5 ErrorCodeToStatusMapper

This is the **one place** that decides which HTTP status a failure maps to. Want `ResourceNotFound` to be `404`? It is decided here, not scattered across controllers.

```
using LiliShop.Application.Common.Results;
using Microsoft.AspNetCore.Http;

namespace LiliShop.Application.Api.ErrorHandling
{
    public static class ErrorCodeToStatusMapper
    {
        private static readonly Dictionary<ErrorCode, int> ErrorCodeToStatusCodeMap =
new()
        {
            // General Errors
            { ErrorCode.GeneralException, StatusCodes.Status500InternalServerError },
            { ErrorCode.InvalidArgument, StatusCodes.Status400BadRequest },
            { ErrorCode.InvalidData, StatusCodes.Status400BadRequest },
            { ErrorCode.SettingError, StatusCodes.Status400BadRequest },
            { ErrorCode.TokenExpired, StatusCodes.Status401Unauthorized },

            // Not Found Errors
            { ErrorCode.ResourceNotFound, StatusCodes.Status404NotFound },
            { ErrorCode.FileNotFound, StatusCodes.Status404NotFound },
            { ErrorCode.UserNotFound, StatusCodes.Status404NotFound },
            { ErrorCode.UserNotFoundForRefreshToken, StatusCodes.Status401Unauthorized },

            // Authorization Errors
            { ErrorCode.AuthorizationRequired, StatusCodes.Status401Unauthorized },
            { ErrorCode.InvalidPassword, StatusCodes.Status401Unauthorized },

            // Conflict Errors
            { ErrorCode.ResourceAlreadyExists, StatusCodes.Status409Conflict },

            // Operation Failures
            { ErrorCode.AddOrUpdateFailed, StatusCodes.Status500InternalServerError },
            { ErrorCode.DeletionFailed, StatusCodes.Status500InternalServerError },
            { ErrorCode.DataFetchFailed, StatusCodes.Status500InternalServerError },
            { ErrorCode.SaveOperationFailed, StatusCodes.Status500InternalServerError },
            { ErrorCode.EmailSendFailed, StatusCodes.Status500InternalServerError },
            { ErrorCode.NotificationSendFailed, StatusCodes.Status500InternalServerError },
            { ErrorCode.UpdateOperationFailed, StatusCodes.Status500InternalServerError },
            { ErrorCode.UpdateFailed, StatusCodes.Status500InternalServerError },
            { ErrorCode.PhotoUploadFailed, StatusCodes.Status500InternalServerError },
            { ErrorCode.MappingFailed, StatusCodes.Status500InternalServerError },
            { ErrorCode.CreationFailed, StatusCodes.Status400BadRequest },

            // External Service Errors
            { ErrorCode.StripeError, StatusCodes.Status500InternalServerError }
        };
    }
}
```

```
public static int GetStatusCode(ErrorCode errorCode)
{
    return ErrorCodeToStatusCodeMap.TryGetValue(errorCode, out var statusCode)
        ? statusCode
        : StatusCodes.Status500InternalServerError; // safe default
}
}
```

5.6 OperationResultHandler

This converts an `OperationResult` into an HTTP response. **This is where the three outcomes become HTTP status codes.** Read it slowly — it is the rulebook for the whole API surface.

```
using LiliShop.Application.Api.ErrorHandling;
using LiliShop.Application.Common.Results;
using LiliShop.Infrastructure.Web.Responses;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using System.Diagnostics;

namespace LiliShop.Infrastructure.Web
{
    /// <summary>
    /// Translates an <see cref="OperationResult"/> coming back from the service layer
    into an HTTP response.
    /// </summary>
    public static class OperationResultHandler
    {
        /// <summary>
        /// Maps a void operation result: failure -> ProblemDetails with the mapped status
        code;
        /// otherwise -> 204 No Content.
        /// </summary>
        public static ActionResult HandleOperationResult(OperationResult? operationResult,
            HttpContext? httpContext)
        {
            if (operationResult is null)
            {
                return BuildProblemDetails(StatusCodes.Status400BadRequest, "The operation
                did not return a result.", httpContext);
            }

            if (operationResult.IsFailure)
            {
                return
                BuildProblemDetails(ErrorCodeToStatusMapper.GetStatusCode(operationResult.ErrorCode ??
                ErrorCode.GeneralException), operationResult.Message, httpContext);
            }
        }
    }
}
```

```

    }

    return new NoContentResult();
}

/// <summary>
/// Maps a data-carrying operation result: failure -> ProblemDetails; an absent
resource
/// (<see cref="OperationResultStatus.NoData"/>) -> 204 No Content; otherwise ->
200 OK with the data.
/// This keeps "not found" off the 404 path so the client can treat it as an empty
outcome rather than
/// an error. Note: only an explicit <c>NoData</c> result yields 204 – a
<c>Success</c> result always
/// carries non-null data (the factory enforces it), so a null payload can never
silently become 204.
/// </summary>
public static ActionResult HandleOperationResult<T>(OperationResult<T>?
operationResult, HttpContext? httpContext)
{
    if (operationResult is null)
    {
        return BuildProblemDetails(StatusCode.Status400BadRequest, "The operation
did not return a result.", httpContext);
    }

    if (operationResult.IsFailure)
    {
        return
BuildProblemDetails(ErrorCodeToStatusMapper.GetStatusCode(operationResult.ErrorCode ??
ErrorCode.GeneralException), operationResult.Message, httpContext);
    }

    if (operationResult.Status == OperationResultStatus.NoData)
    {
        return new NoContentResult();
    }

    return new OkObjectResult(operationResult.Data);
}

private static ObjectResult BuildProblemDetails(int statusCode, string? detail,
HttpContext? httpContext)
{
    string instance = httpContext is null ? string.Empty :
$"{{httpContext.Request.Method}} {{httpContext.Request.Path}}";
    string traceId = Activity.Current?.Id ?? httpContext?.TraceIdentifier ??
string.Empty;

    var problemDetails = new ProblemDetails
    {
        Status = statusCode,

```



```

        Title = new ApiResponse(statusCode, detail).Title,
        Detail = detail,
        Instance = instance
    };

    problemDetails.Extensions["traceId"] = traceId;

    return statusCode == StatusCodes.Status400BadRequest
        ? new BadRequestObjectResult(problemDetails)
        : new ObjectResult(problemDetails) { StatusCode = statusCode };
    }
}
}

```

5.7 BaseApiController

A thin base class so every controller can call the handler with one short method.

```

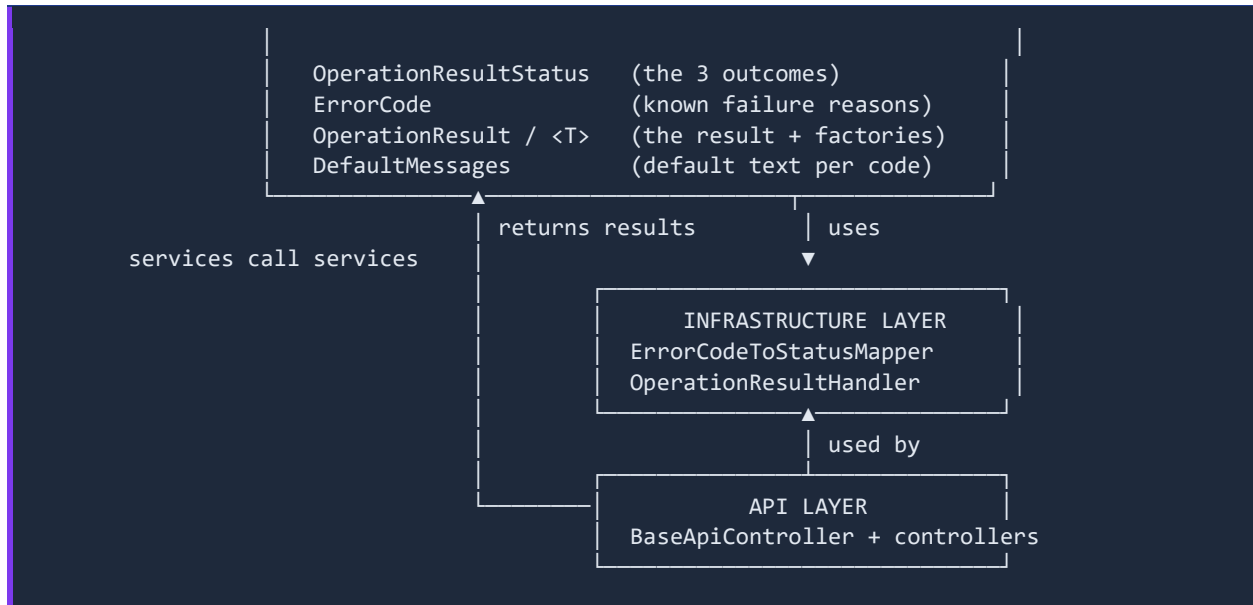
namespace Lili.Shop.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    [ApiConventionType(typeof(MyApiConventions))]
    public class BaseApiController : ControllerBase
    {
        [NonAction]
        protected ActionResult HandleOperationResult(OperationResult operationResult)
            => OperationResultHandler.HandleOperationResult(operationResult,
            this.HttpContext);

        [NonAction]
        protected ActionResult HandleOperationResult<T>(OperationResult<T>?
            operationResult)
            => OperationResultHandler.HandleOperationResult(operationResult,
            this.HttpContext);
    }
}

```

6. Responsibilities at a Glance

APPLICATION LAYER



- **Services** produce results (`Success` / `NoData` / `Failure`) and may consume other services' results.
- **OperationResultHandler** + **ErrorCodeToStatusMapper** translate a result into an HTTP response.
- **Controllers** just call `HandleOperationResult(...)` — they contain no error-mapping logic.

7. How to Use It in Practice

7.1 Creating results in a service

You **never** use `new`. You use the factory methods. Here is a real CRUD service:

```
// Returns DATA, treats "not found" as a 404 error (a dialog-worthy missing resource):
public virtual async Task<OperationResult<ProductBrand>> GetBrandByIdAsync(int brandId)
{
    var productBrand = await _unitOfWork.Repository<ProductBrand>().GetByIdAsync(brandId);

    if (productBrand == null)
    {
        return OperationResult.Failure<ProductBrand>(ErrorCode.ResourceNotFound, "No
        ProductBrand found with the provided id.");
    }

    return OperationResult.Success<ProductBrand>(productBrand);
}

// A VOID operation (no data to return) — success becomes 204:
```

```
public virtual async Task<OperationResult> DeleteBrandAsync(int brandId)
{
    var repo = _unitOfWork.Repository<ProductBrand>();
    ProductBrand entity = await repo.GetByIdAsync(brandId);
    if (entity == null)
    {
        return OperationResult.Failure(ErrorCode.ResourceNotFound, "Brand not found");
    }

    await repo.DeleteAsync(entity);
    int affectedRows = await _unitOfWork.CompleteAsync();

    if (affectedRows <= 0)
    {
        return OperationResult.Failure(ErrorCode.DeletionFailed, "Failed to delete brand.
No rows were affected.");
    }

    return OperationResult.Success();
}
```

A **boolean** result is normal data — `true` and `false` are both valid:

```
public async Task<OperationResult<bool>> GetSubscriptionStatusAsync(int userId, int
productId)
{
    var subscription = await _unitOfWork.Repository<NotificationSubscription>()
        .GetByCriteria(c => c.UserId == userId && c.ProductId == productId && c.IsActive)
        .FirstOrDefaultAsync();

    if (subscription != null)
    {
        return OperationResult.Success<bool>(true);
    }

    return OperationResult.Success<bool>(false, "No active subscription found.");
}
```

7.2 Consuming another service's result (service-to-service)

Because the result carries everything, one service can use another's outcome cleanly — **no casting, no guessing**. Use `IsFailure` to short-circuit, and the `Failure<T>(otherResult)` overload to forward the same error:

```
public virtual async Task<OperationResult<Pagination<DetailedUserDto>>>
GetUsersWithPaginationAsync(UserSpecParams userSpecParams)
{
    if (userSpecParams == null)
    {
        return OperationResult.Failure<Pagination<DetailedUserDto>>(ErrorCode.InvalidData,
"User parameters cannot be null");
    }

    var operationResult = await GetUsersAsync(userSpecParams);

    // Forward the inner failure as-is (keeps its ErrorCode and Message):
    if (operationResult.IsFailure)
    {
        return OperationResult.Failure<Pagination<DetailedUserDto>>(operationResult);
    }

    // Safe to read the data now:
    var data = operationResult.Data.Users;
    // ... build and return Success ...
}
```

Tip: when you specifically need the payload, prefer `HasData`. If `result.HasData` is true, `result.Data` is guaranteed non-null.

7.3 Using results in a controller

Controllers stay tiny. They call the service and hand the result to `HandleOperationResult`:

```
public class ProductBrandController : BaseApiController
{
    private readonly IProductBrandService _productBrandService;

    public ProductBrandController(IProductBrandService productBrandService)
        => _productBrandService = productBrandService;

    [HttpGet("brand/{id}")]
    public async Task<ActionResult<ProductBrand>> GetProductBrandById(int id)
    {
        OperationResult<ProductBrand>? result = await
        _productBrandService.GetBrandByIdAsync(id);
        return HandleOperationResult(result); // 200, 204, or an error – all decided in
one place
    }
}
```

```
[HttpDelete("delete/{id}")]
public async Task<ActionResult> Delete(int id)
{
    var result = await _productBrandService.DeleteBrandAsync(id);
    return HandleOperationResult(result); // 204 on success, error otherwise
}
```

Even an endpoint that used to need lots of manual `if/else` becomes a single line:

```
[HttpGet("product/{productId}")]
public async Task<ActionResult<DiscountDto>> GetSingleDiscountForProduct(int productId)
{
    var result = await _queryService.GetSingleDiscountForProduct(productId);
    // Failure -> mapped error; "no discount for this product" (NoData) -> 204; otherwise
    -> 200.
    return HandleOperationResult(result);
}
```

8. The Tricky Part: null, 0, and Empty Collections

This is the part that confuses most developers. The rule is short:

```
**Success is for *valid data* — even if that data is "empty," 0, or false. NoData is for *a missing resource*. null is never a valid Success payload — it is treated as a bug.**
```

Let's go case by case.

8.1 An empty collection → Success(emptyList) → 200

An empty list is still a **valid answer**. "Show me all discounts" with zero discounts is a successful query that returns `[]`. The collection resource exists; it just has no items.

```
public async Task<OperationResult<Pagination<Discount>>>
GetPaginatedDiscountsAsync(DiscountSpecParams discountParams)
{
    if (discountParams.PageIndex <= 0 || discountParams.PageSize <= 0)
    {
```

```

        return OperationResult.Failure<Pagination<Discount>>(ErrorCode.InvalidData,
            "Invalid pagination parameters. ...");
    }

    var discounts = await _unitOfWork.Repository<Discount>().ListAsync(specs);
    var totalItems = await _unitOfWork.Repository<Discount>().CountAsync(countSpec);

    // An empty page is still a valid collection resource: return 200 with { count: 0,
    data: [] }
    // rather than 204, so the client's pager and data binding work uniformly on the
    success path.
    var paginatedResult = new Pagination<Discount>(discountParams.PageIndex,
        discountParams.PageSize, totalItems, discounts ?? []);
    return OperationResult.Success(paginatedResult);
}

```

Why 200 and not 204? A `204 No Content` has an empty body. Many frontends then receive `null` and must add special handling, which is easy to get wrong. A `200` with `[]` or `{ "count": 0, "data": [] }` is a consistent shape the client can always rely on.

8.2 The number 0 → Success(0) → 200

`0` is data. "How many unread notifications?" → `0` is a real, correct answer, not "missing."

```

return OperationResult.Success(unreadCount); // unreadCount may be 0 – that's fine, it's
still 200

```

8.3 The boolean false → Success(false) → 200

`false` is data. "Is the user subscribed?" → `false` is a real answer.

```

return OperationResult.Success(isSubscribed); // false is valid data, returned as 200

```

8.4 A missing single resource → NoData<T>() → 204

Now the opposite case. You looked up **one specific thing** and it does not exist — and you do **not** want this to look like an error (no `404`, no dialog). Use `NoData`:

```

public async Task<OperationResult<DiscountDto>> GetSingleDiscountForProduct(int productId)
{
    var productDiscount = await _unitOfWork.Context.Set<ProductDiscount>()

```

```

        .Where(pd => pd.ProductId == productId && pd.Discount.DiscountGroupId == null)
        .Select(pd => pd.Discount)
        .FirstOrDefaultAsync();

    if (productDiscount == null)
    {
        // No discount applies to this product – an absent resource, not an error (maps to
204).
        return OperationResult.NoData<DiscountDto>();
    }

    return OperationResult.Success<DiscountDto>(DiscountMapper.MapToDto(productDiscount));
}

```

NoData vs Failure(ResourceNotFound) — which "not found" do I use? Both are valid; the choice is a **business decision**:

- Use **NoData (204)** when "missing" is a **normal** outcome the UI handles silently (e.g. "is there a promotion for this product?").
- Use **Failure(ResourceNotFound) (404)** when the caller asked for something that **should** exist and its absence is a real error worth showing (e.g. opening `/brands/999` directly).

8.5 null data → not allowed (throws)

This is the safety net. If you call `Success<T>(null)`, the factory **throws** `ArgumentNullException`:

```

// ✖ Throws ArgumentNullException – caught immediately, at the line that produced the
bug:
return OperationResult.Success<ProductDto>(null);

// ☑ If "missing" is intended, say so explicitly:
return OperationResult.NoData<ProductDto>();

```

Why throw instead of returning null data? Imagine a mapping fails and silently produces `null`:

```

var dto = mapper.Map<ProductDto>(entity); // bug: returns null
return OperationResult.Success(dto);       // without the guard, this could silently
become a 204

```

If `null` were quietly turned into `204 No Content`, the bug would be **invisible** — the API "works," just returns nothing. By throwing at the source, the mistake surfaces immediately with a clear stack trace, instead of hiding in production. This is the principle of **failing fast**.

8.6 The whole decision in one table

Your value / situation	Factory to use	Status	HTTP	Why
A real object / DTO	<code>Success(dto)</code>	Success	200	Valid data
An empty list []	<code>Success(list)</code>	Success	200	A collection with zero items is valid data
The number 0	<code>Success(0)</code>	Success	200	0 is a valid number
false	<code>Success(false)</code>	Success	200	false is a valid boolean
One specific resource is missing (silent)	<code>NoData<T>()</code>	NoData	204	Absent resource, not an error
Nothing to return from a void action	<code>Success()</code>	Success	204	Succeeded, no body
A known error happened	<code>Failure(code, msg)</code>	Failure	4xx/5xx	Mapped from the error code
Asked-for single resource truly missing & it's an error	<code>Failure(ResourceNotFound)</code>	Failure	404	Caller expected it to exist
<code>null</code> data	✗ (throws)	—	—	Always a bug — use <code>NoData</code> if absence is intended

9. How Results Become HTTP Responses

Putting the handler's logic into plain words:

For a data result `OperationResult<T>`:

1. `result is null?` → `400 Bad Request`. *(A `null` result means the service itself misbehaved.)*
2. `result.IsFailure?` → look up the HTTP status from `ErrorCode` and return a `ProblemDetails` body.
3. `result.Status == NoData?` → `204 No Content`.
4. Otherwise (Success) → `200 OK` with `result.Data`.

For a void result `OperationResult`:

1. `result is null?` → `400 Bad Request`.
2. `result.IsFailure?` → mapped error + `ProblemDetails`.
3. Otherwise (Success or NoData) → `204 No Content`.

Failures return a standard `ProblemDetails` object (with `status`, `title`, `detail`, `instance`, and a `traceId`), so every error in the API looks the same to clients.

10. Quick Reference Cheat Sheet

```
// ---- PRODUCE (inside a service) ----

// Data succeeded:
return OperationResult.Success(dto);           // 200 + body
return OperationResult.Success(list);          // 200 + [] (even if empty)
return OperationResult.Success(0);             // 200 + 0
return OperationResult.Success(false);         // 200 + false

// Void succeeded:
return OperationResult.Success();              // 204

// A single resource is missing, but that's OK:
return OperationResult.NoData<ProductDto>();   // 204

// Something failed:
return OperationResult.Failure(ErrorCode.DeletionFailed); // void failure
return OperationResult.Failure<ProductDto>(ErrorCode.ResourceNotFound, "Not found."); // typed failure

// Forward another service's failure unchanged:
if (inner.IsFailure) return OperationResult.Failure<ProductDto>(inner);
```

```
// ---- CONSUME (service-to-service) ----

var result = await _other.DoSomethingAsync();

if (result.IsFailure)           { /* handle / forward result.ErrorCode, result.Message */ }
if (result.HasData)             { var value = result.Data; /* guaranteed non-null */ }
if (result.Status == OperationResultStatus.NoData) { /* nothing to show */ }

// ---- CONSUME (controller) ----

return HandleOperationResult(result); // does everything above, automatically
```

11. Common Mistakes (FAQ)

Q: I returned an empty list as `NoData` and the frontend broke. Why? An empty list is valid data → use `Success(list)` (200). `NoData` (204) sends an empty body, which the client may read as `null`.

Q: `Success(0)` — won't `0` be treated as "nothing"? No. `0` and `false` are values, not `null`. Only `null` is rejected. They return `200` with the value.

Q: My `Success(...)` threw `ArgumentNullException`. Is that a bug in the pattern? No — that is the pattern protecting you. Your data was `null`. Either fix why it's `null`, or, if absence is intended, return `NoData<T>()`.

Q: When do I use `IsSuccess` vs `HasData`? `IsSuccess` = "did not fail" (true for `Success` and `NoData`). `HasData` = "succeeded with a payload" (true only for `Success`). If you are about to read `Data`, check `HasData`.

Q: Should "not found" be 404 or 204? It depends on the business meaning. Silent/optional absence → `NoData` (204). A resource the caller expected to exist → `Failure(ResourceNotFound)` (404). See [section 8.4](#).

Q: Do I ever use `new OperationResult(...)`? No. Always use the factory methods. The constructors are not public on purpose.

12. Summary

- The **Result Pattern** returns a small object describing an outcome instead of relying on exceptions

for expected situations.

- There are exactly **three outcomes: Success, NoData, Failure**.
- One object type (`OperationResult` / `OperationResult<T>`) is used **everywhere** — service-to-service

and service-to-API — created only through **factory methods**.

- **Valid data is always Success** — including empty lists, `0`, and `false` (all `200`).
- **NoData means a resource is genuinely absent** (`204`), so "not found" doesn't have to be a `404`.
- **null is never a valid Success** — the factory throws, so bugs fail fast instead of hiding.
- The API layer (`OperationResultHandler` + `ErrorCodeToStatusMapper`) is the **single place** that turns

outcomes into HTTP responses, keeping controllers tiny and behavior consistent.

This makes the system **explicit, safe, predictable, and easy to test** — for both backend and frontend developers.